



Pauta Interrogación 1

10 de Septiembre 2026

Índice

Pregunta 1. Misceláneo (6 puntos) [30 % nota final]	2
Forma A	2
Parte A: Pauta de corrección (2 punto)	2
Parte B: Pauta de corrección (2 punto)	2
Parte C: Pauta de corrección (2 punto)	3
Forma B	4
Parte A: Pauta de corrección (2 punto)	4
Parte B: Pauta de corrección (2 punto)	4
Parte C: Pauta de corrección (2 punto)	5
Pregunta 2. Bingo (6 puntos) [30 % nota final]	6
Forma A	6
Parte A: Pauta de corrección (4 puntos)	6
Parte B: Pauta de corrección (2 puntos)	6
Forma B	7
Parte A: Pauta de corrección (4 puntos)	7
Parte B: Pauta de corrección (2 puntos)	7
Pregunta 3. Sistema TAG (6 puntos) [40 % nota final]	8
Forma A	8
Parte A: Diagrama de clases UML y justificación (2 puntos)	8
Parte B: Función <code>procesar_lectura</code> (2.5 puntos)	10
Parte C: Función <code>consultar_patentes</code> (1.5 puntos)	11
Forma B	13
Parte A: Diagrama de clases UML y justificación (2 puntos)	13
Parte B: Función <code>resumir_pagos</code> (2.5 puntos)	13
Parte C: Función <code>consultar_montos</code> (1.5 puntos)	14

Pregunta 1. Misceláneo (6 puntos) [30 % nota final]

Forma A

Parte A: Pauta de corrección (2 punto)

Parte A.1 (1 punto)

La instrucción `open("jugadores.csv")` utiliza una ruta relativa, por lo que Python busca el archivo a partir del directorio de trabajo actual. Como la terminal se encuentra en `iic2233`, Python intenta encontrar el archivo en dicho directorio, pero `jugadores.csv` se encuentra dentro de `datos`.

- **0,5 punto:** Identifica que el problema proviene de la ruta. `jugadores.csv`.
- **0,5 puntos:** Justifica correctamente por qué la ruta utilizada es la causa del problema.

Parte A.2 (1 punto)

Una posible secuencia de comandos es:

```
cd datos
python cargar.py
```

- **1,0 punto:** Entrega una secuencia válida de comandos que permite ejecutar `cargar.py` desde un directorio de trabajo en el que `open("jugadores.csv")` pueda encontrar correctamente el archivo.

Nota: No se debe exigir una secuencia específica de comandos. Se considera correcta cualquier alternativa que, respetando la estructura entregada y sin modificar el código de `cargar.py`, permita que `open("jugadores.csv")` encuentre el archivo.

Parte B: Pauta de corrección (2 punto)

Parte B.1 (1 punto)

La segunda versión utiliza menos memoria porque el generador entrega las líneas una a una, por lo que no es necesario mantener todas las líneas del archivo simultáneamente en memoria. En cambio, `readlines()` construye una lista que contiene todas las líneas del archivo antes de procesarlas.

- **0,5 punto:** Reconoce correctamente que el generador utiliza menos memoria. `readlines()`.
- **0,5 puntos:** Justifica correctamente y de manera completa la diferencia en el uso de memoria entre las dos formas presentadas.

Parte B.2 (1 punto)

No se mantiene de la misma forma la ventaja de memoria. Para ordenar todas las líneas según su contenido es necesario disponer de ellas antes de determinar su orden, por lo que deben almacenarse antes de realizar el ordenamiento.

- **0,5 puntos:** Indica correctamente que la ventaja en el uso de memoria no se mantiene.
- **0,5 puntos:** Justifica correctamente que, al ser necesario disponer de toda la información antes de realizar el ordenamiento, la ventaja aparente en el uso de memoria deja de existir.

Parte C: Pauta de corrección (2 punto)

Parte C.1 (2/3 punto)

La expresión genera un error. Las instancias de una `namedtuple` son inmutables, por lo que no es posible modificar el valor de uno de sus atributos después de haber creado la instancia.

- **1/3 puntos:** Indica correctamente que se genera un error.
- **1/3 puntos:** Explica que sucede porque la estructura `namedtuple` es inmutable

Parte C.2 (2/3 punto)

Una expresión posible es:

```
inscritos - asistentes
```

Esta operación entrega los elementos que pertenecen a `inscritos` pero no a `asistentes`.

- **2/3 puntos:** Presenta una expresión válida y funcional que responde correctamente a lo solicitado en el enunciado.

Nota de corrección: No se debe exigir una expresión específica. Por ejemplo, tanto

```
inscritos - asistentes
```

como

```
inscritos.difference(asistentes)
```

son respuestas correctas.

Parte C.3 (2/3 punto)

La expresión genera un error. Los objetos de tipo `set` no permiten acceder a sus elementos mediante índices, por lo que no es posible utilizar `inscritos[0]`.

- **1/3 puntos:** Indica correctamente que la expresión genera un error
- **1/3 puntos:** Indica que es debido a que los elementos de un `set` no son accesibles mediante índices

Forma B

Parte A: Pauta de corrección (2 punto)

Parte A.1 (1 punto)

La instrucción `open("partidas.csv")` utiliza una ruta relativa, por lo que Python busca el archivo a partir del directorio de trabajo actual. Como la terminal se encuentra en **Escritorio**, Python intenta encontrar el archivo en dicho directorio, pero `partidas.csv` se encuentra dentro de `iic2233/registros`.

- **0,5 punto:** Identifica que el problema proviene de la ruta.
- **0,5 puntos:** Justifica correctamente por qué la ruta utilizada es la causa del problema.

Parte A.2 (1 punto)

Una posible secuencia de comandos es:

```
cd iic2233/registros
python procesar.py
```

- **1,0 punto:** Propone correctamente una serie de comandos que permiten solucionar el problema.

Nota: No se debe exigir una secuencia específica de comandos. Se considera correcta cualquier alternativa que, respetando la estructura entregada y sin modificar el código de `procesar.py`, permita que `open("partidas.csv")` encuentre el archivo.

Parte B: Pauta de corrección (2 punto)

Parte B.1 (1 punto)

La segunda versión utiliza menos memoria porque el generador entrega las líneas una a una, por lo que no es necesario mantener todas las líneas del archivo simultáneamente en memoria. En cambio, `readlines()` construye una lista que contiene todas las líneas del archivo antes de procesarlas.

- **0,5 punto:** Reconoce correctamente que el generador utiliza menos memoria.
- **0,5 puntos:** Justifica correctamente y de manera completa la diferencia en el uso de memoria entre las dos formas presentadas.

Parte B.2 (1 punto)

No se mantiene de la misma forma la ventaja de memoria. Para ordenar todas las temperaturas de menor a mayor es necesario disponer de ellas antes de determinar su orden, por lo que deben almacenarse antes de realizar el ordenamiento.

- **0,5 punto:** Indica correctamente que la ventaja en el uso de memoria no se mantiene.
- **0,5 puntos:** Justifica correctamente que, al ser necesario disponer de toda la información antes de realizar el ordenamiento, la ventaja aparente en el uso de memoria deja de existir.

Parte C: Pauta de corrección (2 punto)

Parte C.1 (2/3 punto)

La expresión genera un error. Las instancias de una `namedtuple` son inmutables, por lo que no es posible modificar uno de sus elementos mediante asignación por índice después de haber creado la instancia.

- **1/3 puntos:** Indica correctamente que se genera un error.
- **1/3 puntos:** Menciona que se debe porque la estructura `namedtuple` es inmutable.

Parte C.2 (2/3 punto)

Una expresión posible es:

`prestados & devueltos`

Esta operación entrega los elementos que pertenecen tanto a `prestados` como a `devueltos`.

- **2/3 puntos:** Escribe una expresión válida que obtiene los elementos que se encuentran tanto en `prestados` como en `devueltos`.

Nota: No se debe exigir una expresión específica. Por ejemplo, tanto

`prestados & devueltos`

como

`prestados.intersection(devueltos)`

son respuestas correctas.

Parte C.3 (2/3 punto)

La expresión genera un error. Los objetos de tipo `set` no permiten acceder a sus elementos mediante índices ni realizar *slicing*, por lo que no es posible utilizar `prestados[:2]`.

- **1/3 puntos:** Indica correctamente que la expresión genera un error.
- **1/3 puntos:** Justifica que es debido a que los elementos de un `set` no son accesibles mediante índices.

Pregunta 2. Bingo (6 puntos) [30 % nota final]

Forma A

Parte A: Pauta de corrección (4 puntos)

```
1 class CreadorCartones:
2     def __init__(self, total_cartones: int):
3         # 2 puntos: usa sets o diccionarios para guardar los cartones
4         self.cartones = set()
5         self.total_cartones = total_cartones
6         self.generar_cartones()
7
8     def generar_cartones(self) -> None:
9         # 1 punto: se asegura de llegar a la cantidad de
10        # cartones pedida en caso de repeticiones
11        while len(cartones) < self.total_cartones:
12            # 1 punto: usa tuplas para representar cada carton
13            numeros_random = tuple(generar_numeros())
14            self.cartones.add(numeros_random)
```

Parte B: Pauta de corrección (2 puntos)

- (1 punto) Tuplas: Mencionar que las tuplas son **inmutables** o que son **hasheables**, por lo que se pueden insertar en un set o usar como *key* en un diccionario.
- (1 punto) Set (o diccionario): Los sets permiten rápidamente revisar si un elemento ya existe. Si se agrega un elemento repetido, el tamaño del set no cambia y se queda igual.

Forma B

Parte A: Pauta de corrección (4 puntos)

```
1 class Registro:
2     """
3     Esta solucion usa un diccionario pero usando un default dict es mejor
4     Se simplifican los 2 métodos del registro.
5     """
6     def __init__(self):
7         # 2 puntos: Usar un diccionario o default dict
8         self.registro = {}
9
10    def registrar_numeros(self, id_persona: str, carton: tuple) -> None:
11        # 1 punto: Insertar correctamente incluso en caso de que
12        # no hayan cartones registrados al usuario aun
13        if not id_persona in self.registro:
14            self.registro[id_persona] = []
15        self.registro[id_persona].append(carton)
16
17
18    def buscar_numeros(self, id_persona: str) -> list:
19        # 1 punto: retornar la lista correctamente y
20        # lista vacia en caso que no existan cartones registrados
21        if not id_persona in self.registro:
22            return []
23        return self.registro[id_persona]
```

Parte B: Pauta de corrección (2 puntos)

- Diccionarios (o default dict) (2 puntos): Los diccionarios permiten insertar (1 punto) y buscar datos (1 punto) de manera eficiente asociados a un *key*.

Pregunta 3. Sistema TAG (6 puntos) [40 % nota final]

Distribución de puntaje

Criterio de corrección. En cada ítem se indica el tipo de corrección que debe utilizarse:

- **Corrección binaria:** se asigna cero o el puntaje completo del criterio.
- **Corrección ternaria:** se asigna cero, la mitad o el puntaje completo del criterio, según el grado de cumplimiento.

Se acepta cualquier pseudocódigo claro, consistente e independiente de un lenguaje de programación particular. No se exige que los mensajes mostrados coincidan textualmente con los de esta pauta, siempre que permitan identificar el problema ocurrido.

Nota de formato: en esta pauta se utiliza el entorno `python` solamente para obtener tipografía monoespaciada, colores e indentación similares a los utilizados en la AC2. El contenido de esos bloques es pseudocódigo y no código Python: por ello se emplean palabras como `intentar`, `capturar`, `para cada`, `levantar`, `entregar` y `retornar`.

Forma A

Parte A: Diagrama de clases UML y justificación (2 puntos)

Las siguientes son dos modelaciones posibles. Ambas son válidas. Pueden existir otras soluciones correctas si representan la información y el comportamiento pedidos de manera consistente.

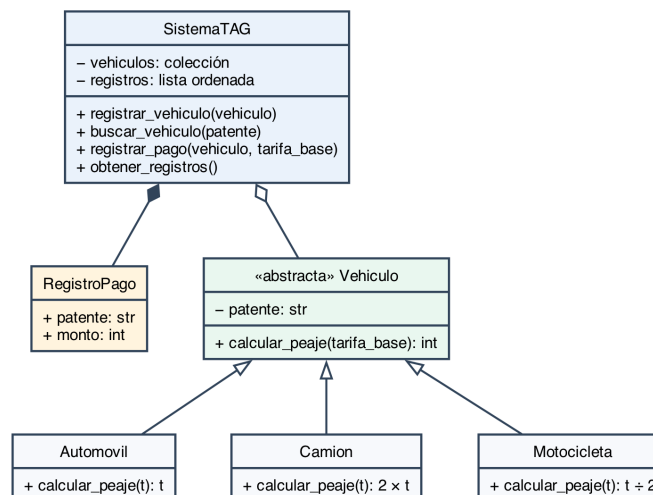


Figura 1: Modelación mediante herencia y polimorfismo.

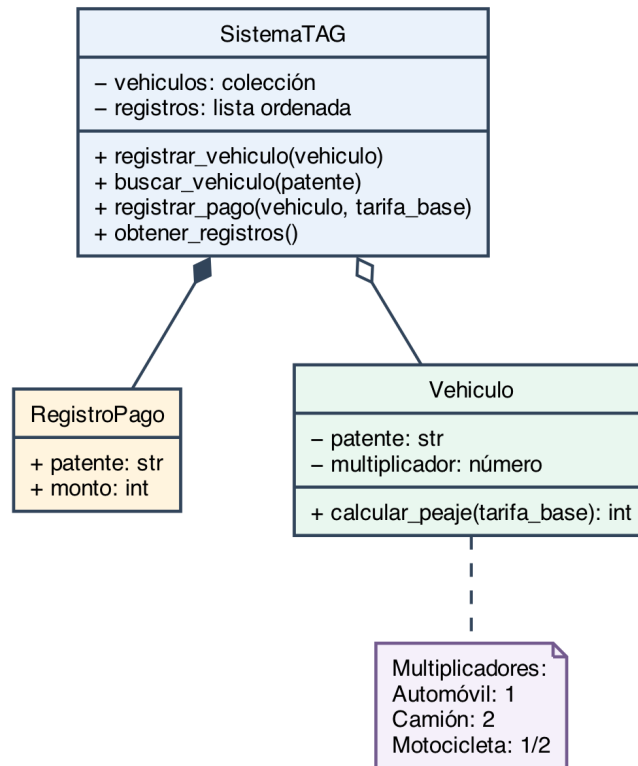


Figura 2: Modelación mediante una única clase y un multiplicador.

No se exige herencia: se acepta cualquier modelación que represente los tres tipos, mantenga `calcular_peaje` en el vehículo y conserve los pagos en orden. La composición también puede reemplazarse por una asociación o agregación consistente.

Justificación posible. El sistema solicita el cálculo mediante una operación común; cada vehículo determina el monto que le corresponde.

Distribución de puntaje:

- **Clases y atributos (0.5 puntos, corrección ternaria).**
 - 0.5:** Incluye las clases centrales, sus atributos y los tres tipos de vehículo.
 - 0.25:** Omite una clase, atributo o tipo relevante.
 - 0:** No permite representar vehículos y pagos.
- **Operaciones y relaciones (0.5 puntos, corrección ternaria).**
 - 0.5:** Incluye las operaciones y representa múltiples vehículos y registros ordenados.
 - 0.25:** Las operaciones o las relaciones están incompletas.
 - 0:** No representa ninguna de ellas correctamente.
- **Cálculo del peaje (0.5 puntos, corrección ternaria).**
 - 0.5:** Modela la operación y los tres cobros correctamente.
 - 0.25:** La operación existe, pero un cobro es incorrecto o su ubicación es ambigua.

0: No modela el cálculo pedido.

■ **Justificación (0.5 puntos, corrección ternaria).**

0.5: Explica la operación común y el cálculo propio de cada vehículo.

0.25: Solo menciona que los tipos pagan montos distintos.

0: No justifica o contradice su diagrama.

Parte B: Función procesar_lectura (2.5 puntos)

Una posible solución en pseudocódigo es:

```
1 procesar_lectura(sistema, lectura):
2     intentar:
3         partes <- separar(lectura, ";")
4
5         si largo(partes) != 2 o partes[0] está vacía:
6             levantar ValueError
7
8         patente <- partes[0]
9         tarifa_base <- convertir_a_entero(partes[1])
10
11         si tarifa_base <= 0:
12             levantar ValueError
13
14         vehiculo <- sistema.buscar_vehiculo(patente)
15         retornar sistema.registrar_pago(vehiculo, tarifa_base)
16
17     capturar ValueError:
18         mostrar "Lectura o tarifa inválida"
19
20     capturar KeyError:
21         mostrar "Patente no registrada"
22
23     retornar nulo
```

Distribución de puntaje:

■ **Estructura de la lectura (0.5 puntos, corrección ternaria).**

0.5: Separa o analiza la lectura usando “;”, comprueba que existan exactamente dos componentes y que la patente no esté vacía.

0.25: Separa o analiza la lectura usando “;”, pero comprueba solo una de estas condiciones: que existan exactamente dos componentes o que la patente no esté vacía.

0: No interpreta la lectura usando “;” o utiliza sus componentes sin comprobar la cantidad ni la patente vacía.

■ **Tarifa (0.5 puntos, corrección ternaria).**

0.5: Convierte a entero y exige un valor positivo.

0.25: Realiza solo una de ambas acciones.

0: No convierte ni valida.

■ **Caso exitoso (0.5 puntos, corrección ternaria).**

0.5: Busca, registra y retorna el registro.

0.25: Realiza correctamente dos de esas acciones.

0: No registra correctamente un pago válido.

■ **Manejo de ValueError (0.5 puntos, corrección ternaria).**

0.5: Captura específicamente, muestra un mensaje y retorna None sin registrar.

0.25: Captura específicamente, pero omite el mensaje o el retorno.

0: No captura, usa solo `Exception` o registra.

■ **Manejo de KeyError (0.5 puntos, corrección ternaria).**

0.5: Captura específicamente, muestra un mensaje y retorna None sin registrar.

0.25: Captura específicamente, pero omite el mensaje o el retorno.

0: No captura, usa solo `Exception` o registra.

Es válido dejar que `registrar_pago` detecte una tarifa menor o igual a cero, pues la operación entregada levanta `ValueError` antes de registrar el pago. Si se captura únicamente `Exception`, no se cumple la exigencia de distinguir las excepciones específicas.

Parte C: Función `consultar_patentes` (1.5 puntos)

Una posible solución en pseudocódigo es:

```
1  consultar_patentes(sistema, monto_minimo):  
2      registros <- sistema.obtener_registros()  
3  
4      para cada registro en registros:  
5          si registro.monto >= monto_minimo:  
6              entregar registro.patente
```

Una posible solución en Python (para que se guiarse en la corrección) es:

```
1  def consultar_patentes(sistema, monto_minimo):  
2      return (  
3          registro.patente  
4          for registro in sistema.obtener_registros()  
5          if registro.monto >= monto_minimo  
6      )
```

Distribución de puntaje:

■ **Recorrido y filtro (0.5 puntos, corrección ternaria).**

0.5: Recorre todo y aplica $\geq$.

0.25: Recorre, pero filtra de forma incompleta o usa $>$.

0: No hay recorrido o filtro utilizable.

- **Entrega diferida (0.5 puntos, corrección binaria).**

0.5: Entrega cada patente una por una, sin almacenarlas.

0: Retorna una colección, termina en la primera o entrega otro dato.

- **Orden y repeticiones (0.5 puntos, corrección ternaria).**

0.5: Conserva ambas propiedades.

0.25: Conserva solo una.

0: No conserva ninguna.

Una solución que construye y retorna una lista no obtiene el criterio binario asociado a la entrega uno por uno, aunque puede obtener los demás criterios. Usar **RETORNAR** dentro del ciclo tampoco equivale a **ENTREGAR**, ya que finalizaría la consulta después del primer resultado.

Forma B

Parte A: Diagrama de clases UML y justificación (2 puntos)

Se aceptan los mismos diagramas y alternativas de la Forma A.

Justificación posible. La lista de registros admite pagos repetidos para una patente y conserva el orden de inserción.

Distribución de puntaje:

- **Clases y atributos (0.5 puntos):** aplicar el criterio de la Forma A.
- **Operaciones y relaciones (0.5 puntos):** aplicar el criterio de la Forma A.
- **Cálculo del peaje (0.5 puntos):** aplicar el criterio de la Forma A.
- **Justificación (0.5 puntos, corrección ternaria).**

0.5: Explica tanto la conservación de pagos repetidos como el orden de registro.

0.25: Explica solamente una de ambas propiedades.

0: No justifica o contradice su diagrama.

Parte B: Función resumir_pagos (2.5 puntos)

Una posible solución en pseudocódigo es:

```
1 resumir_pagos(sistema, patente):
2     intentar:
3         sistema.buscar_vehiculo(patente)
4
5     capturar KeyError:
6         mostrar "Patente no registrada"
7         retornar nulo
8
9     cantidad <- 0
10    total <- 0
11    registros <- sistema.obtener_registros()
12
13    para cada registro en registros:
14        si registro.patente = patente:
15            cantidad <- cantidad + 1
16            total <- total + registro.monto
17
18    retornar cantidad, total
```

La búsqueda inicial permite distinguir un vehículo registrado que todavía no posee pagos, para el cual se retorna (0,0), de una patente que no está registrada, para la cual se informa el problema y se retorna None.

Distribución de puntaje:

- **Validación de patente (0.5 puntos, corrección binaria).**

0.5: Usa buscar_vehiculo.

0: No valida o infiere la existencia desde los pagos.

■ **Manejo de KeyError (0.5 puntos, corrección ternaria).**

0.5: Captura específicamente, muestra un mensaje y retorna None.

0.25: Captura específicamente, pero omite el mensaje o el retorno.

0: No captura o usa solo Exception.

■ **Preparación y recorrido (0.5 puntos, corrección ternaria).**

0.5: Inicializa ambos acumuladores y recorre todo.

0.25: Una de ambas partes está incompleta.

0: No prepara ni recorre los datos correctamente.

■ **Cálculo del resumen (0.5 puntos, corrección ternaria).**

0.5: Filtra, cuenta y suma correctamente.

0.25: Filtra, pero solo cuenta o suma correctamente.

0: No filtra o no calcula resultados.

■ **Retorno (0.5 puntos, corrección ternaria).**

0.5: Retorna ambos valores, incluido (0,0).

0.25: Maneja solo el caso con pagos o solo el caso sin pagos.

0: No retorna los dos resultados.

Revisar únicamente los registros no reemplaza la llamada a `buscar_vehiculo`: de esa forma no sería posible distinguir una patente inexistente de un vehículo registrado sin pagos.

Parte C: Función `consultar_montos` (1.5 puntos)

Una posible solución en pseudocódigo es:

```
1  consultar_montos(sistema, patente):
2      registros <- sistema.obtener_registros()
3
4      para cada registro en registros:
5          si registro.patente = patente:
6              entregar registro.monto
```

Una posible solución en Python (para que se guiarse en la corrección) es:

```
1  def consultar_montos(sistema, patente):
2      return (
3          registro.monto
4          for registro in sistema.obtener_registros()
5          if registro.patente == patente
6      )
```

Distribución de puntaje:

- **Recorrido y filtro (0.5 puntos, corrección ternaria).**

0.5: Recorre todo y filtra por igualdad de patente.

0.25: Recorre, pero el filtro es incompleto.

0: No hay recorrido o filtro utilizable.

- **Entrega diferida (0.5 puntos, corrección binaria).**

0.5: Entrega cada monto uno por uno, sin almacenarlos.

0: Retorna una colección, termina en el primero o entrega otro dato.

- **Orden y repeticiones (0.5 puntos, corrección ternaria).**

0.5: Conserva ambas propiedades.

0.25: Conserva solo una.

0: No conserva ninguna.

No es necesario llamar a `buscar_vehiculo`, ya que la especificación indica que, si no existen pagos asociados a la patente, la consulta simplemente no debe entregar resultados.